



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

100+ Interview Questions : Design Patterns (Singleton, Factory, Strategy, Repository, Unit of Work, Dependency Injection) and SOLID Principles

Q&A on the Singleton pattern

1. What is the Singleton pattern and why is it used?

Answer:

The Singleton pattern ensures a class has only one instance and provides a global point of access to it. It's commonly used when exactly one object is needed to coordinate actions across the system, such as configuration settings, logging, or caching.

2. How do you implement a thread-safe Singleton in .NET?

Answer:

One common thread-safe implementation uses **lazy initialization** with `Lazy<T>`:

```
public sealed class Singleton
```



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

```
{  
    private static readonly Lazy<Singleton> instance = new  
    Lazy<Singleton>(() => new Singleton());  
  
    private Singleton() { }  
  
    public static Singleton Instance => instance.Value;  
}
```

This approach ensures thread safety and lazy initialization without locks.

3. What are the common pitfalls of the Singleton pattern?

Answer:

- **Global state:** Singleton can lead to hidden dependencies and make testing difficult.
 - **Tight coupling:** Other classes depend on the Singleton instance, reducing flexibility.
 - **Concurrency issues:** If not implemented thread-safe, it can cause race conditions.
 - **Resource contention:** Singleton might become a bottleneck if overused in concurrent environments.
-

4. How does the Singleton pattern affect unit testing?

Answer:

Singletons can hinder unit testing because they introduce global state, making tests dependent on a shared instance. This can cause tests to be flaky or order-dependent. To mitigate this, use interfaces and dependency injection, or design the Singleton to allow resetting its state for tests.

5. Can you explain the difference between lazy and eager initialization in Singleton?



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

Answer:

- **Eager Initialization:** The Singleton instance is created at the time of class loading. It's simple but can waste resources if the instance is never used.
- **Lazy Initialization:** The instance is created only when it is first accessed. It saves resources but requires careful implementation for thread safety.

Factory pattern Q&A

6. What is the Factory pattern and when would you use it?

Answer:

The Factory pattern is a creational design pattern that provides an interface for creating objects but allows subclasses or implementations to decide which class to instantiate. It's used to encapsulate object creation, promoting loose coupling and flexibility when the exact types of objects aren't known until runtime.

Use case: When a class can't anticipate the class of objects it needs to create, or when you want to delegate responsibility for object creation to subclasses.

7. Explain the difference between Factory Method and Abstract Factory patterns.

Answer:

- **Factory Method:** Defines an interface for creating an object but lets subclasses decide which class to instantiate. It uses inheritance and relies on subclass overriding.
 - **Abstract Factory:** Provides an interface to create families of related or dependent objects without specifying their concrete classes. It uses composition and is useful when you need to create multiple related objects together.
-



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

8. How do you implement a Factory pattern in C#?

Answer:

A simple example of Factory Method:

```
// Product interface
public interface IAnimal
{
    void Speak();
}

// Concrete Products
public class Dog : IAnimal
{
    public void Speak() => Console.WriteLine("Woof");
}

public class Cat : IAnimal
{
    public void Speak() => Console.WriteLine("Meow");
}

// Factory
public class AnimalFactory
{
    public static IAnimal CreateAnimal(string animalType)
    {
        return animalType.ToLower() switch
        {
            "dog" => new Dog(),
            "cat" => new Cat(),
            _ => throw new ArgumentException("Invalid animal type")
        };
    }
}
```

Usage:



```
var dog = AnimalFactory.CreateAnimal("dog");  
dog.Speak(); // Outputs: Woof
```

9. What are the advantages of using the Factory pattern?

Answer:

- **Encapsulates object creation:** Decouples client code from concrete classes.
 - **Promotes code reuse:** Centralizes object creation logic.
 - **Enhances maintainability:** Adding new types requires minimal changes to existing code.
 - **Supports polymorphism:** Clients work with interfaces or base classes rather than concrete types.
 - **Improves testability:** Can easily mock or swap factory implementations.
-

10. Can Factory pattern help with Dependency Injection? Provide example.

Answer:

Yes! Factory pattern can complement Dependency Injection (DI) by abstracting complex object creation logic, especially when the creation involves runtime parameters or complex setup that DI containers can't handle easily.

Example: Imagine a service that needs different data repositories based on a runtime parameter.

```
public interface IRepository { void Save(); }  
public class SqlRepository : IRepository { public void Save() =>  
    Console.WriteLine("Saving to SQL DB"); }  
public class InMemoryRepository : IRepository { public void Save()  
=> Console.WriteLine("Saving in Memory"); }
```

```
public interface IRepositoryFactory
```



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

```
{
    IRepository CreateRepository(string repoType);
}

public class RepositoryFactory : IRepositoryFactory
{
    public IRepository CreateRepository(string repoType)
    {
        return repoType.ToLower() switch
        {
            "sql" => new SqlRepository(),
            "memory" => new InMemoryRepository(),
            _ => throw new ArgumentException("Invalid repository
type")
        };
    }
}

// Consumer class with DI
public class Service
{
    private readonly IRepositoryFactory _repositoryFactory;

    public Service(IRepositoryFactory repositoryFactory)
    {
        _repositoryFactory = repositoryFactory;
    }

    public void SaveData(string repoType)
    {
        var repo = _repositoryFactory.CreateRepository(repoType);
        repo.Save();
    }
}
```

Here, DI injects the `IRepositoryFactory` while the factory manages object creation based on runtime input. This promotes loose coupling and flexibility.



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

Strategy Design Pattern

11. What is the Strategy pattern and what problem does it solve?

Answer:

The Strategy pattern is a behavioral design pattern that defines a family of algorithms, encapsulates each one, and makes them interchangeable at runtime.

It allows the algorithm to vary independently from clients that use it.

Problem it solves:

Avoids large `if-else` or `switch` statements when selecting behavior and promotes flexibility by decoupling the algorithm from the client using it.

12. How would you implement Strategy pattern in .NET?

Answer:

Example: Payment strategy selection

```
// Strategy Interface
public interface IPaymentStrategy
{
    void Pay(decimal amount);
}

// Concrete Strategies
public class CreditCardPayment : IPaymentStrategy
{
    public void Pay(decimal amount) => Console.WriteLine($"Paid {amount} using Credit Card");
}

public class PayPalPayment : IPaymentStrategy
{
    public void Pay(decimal amount) => Console.WriteLine($"Paid {amount} using PayPal");
}
```



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

```
}

// Context
public class PaymentContext
{
    private IPaymentStrategy _paymentStrategy;

    public PaymentContext(IPaymentStrategy paymentStrategy)
    {
        _paymentStrategy = paymentStrategy;
    }

    public void ExecutePayment(decimal amount)
    {
        _paymentStrategy.Pay(amount);
    }
}
```

Usage:

```
var context = new PaymentContext(new PayPalPayment());
context.ExecutePayment(200); // Outputs: Paid 200 using PayPal
```

13. What are real-world examples of Strategy pattern usage?

Answer:

- **Payment gateways** (Credit Card, PayPal, UPI, etc.)
- **Sorting algorithms** (QuickSort, MergeSort, BubbleSort)
- **Authentication strategies** (OAuth, JWT, LDAP)
- **Compression algorithms** (ZIP, RAR, TAR)
- **Loggers** (FileLogger, ConsoleLogger, DatabaseLogger)



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

14. How does the Strategy pattern promote Open/Closed Principle?

Answer:

It promotes the **Open/Closed Principle** by allowing you to add new strategies (algorithms or behaviors) without modifying the existing code.

The context class uses an interface for the strategy, so new behavior can be added just by creating a new class that implements the interface—no need to touch existing logic.

15. How does Strategy pattern differ from State pattern?

Answer:

Aspect	Strategy Pattern	State Pattern
Purpose	Encapsulates interchangeable behaviors (algorithms).	Encapsulates states and transitions between them.
Client Control	Client decides which strategy to use.	Object changes its own state internally.
Behavior Switch	Switched externally (e.g., passed as a parameter).	Switched internally (e.g., via method call).
Example	Payment method selection.	Document lifecycle (Draft → Published → Archived).

Repository Design Pattern

16. What is the Repository pattern and why is it important in .NET applications?

Answer:

The Repository pattern abstracts the data access layer from the business logic by providing a collection-like interface to access domain objects.



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

It helps keep data access logic centralized and makes the codebase easier to maintain, test, and swap out data sources (e.g., switching from EF Core to Dapper or an API).

17. How do you implement Repository pattern with Entity Framework?

Answer:

Here's a basic example:

```
// Entity
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
}

// Generic Repository Interface
public interface IRepository<T> where T : class
{
    Task<IEnumerable<T>> GetAllAsync();
    Task<T> GetByIdAsync(int id);
    Task AddAsync(T entity);
    void Update(T entity);
    void Delete(T entity);
    Task SaveAsync();
}

// EF Core implementation
public class Repository<T> : IRepository<T> where T : class
{
    private readonly DbContext _context;
    private readonly DbSet<T> _dbSet;

    public Repository(DbContext context)
    {
        _context = context;
        _dbSet = context.Set<T>();
    }
}
```



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

```
public async Task<IEnumerable<T>> GetAllAsync() => await
_dbSet.ToListAsync();
public async Task<T> GetByIdAsync(int id) => await
_dbSet.FindAsync(id);
public async Task AddAsync(T entity) => await
_dbSet.AddAsync(entity);
public void Update(T entity) => _dbSet.Update(entity);
public void Delete(T entity) => _dbSet.Remove(entity);
public async Task SaveAsync() => await
_context.SaveChangesAsync();
}
```

18. What are the benefits of using Repository pattern?

Answer:

- **Separation of concerns** between business and data access layers
 - **Improved testability** (can mock repositories)
 - **Centralized query logic** for maintainability
 - **Easier to switch persistence implementations**
 - **Promotes cleaner and more organized code**
-

19. How do you handle complex queries in the Repository pattern?

Answer:

- Add **custom methods** in a specialized repository interface (e.g., `IProductRepository`)
- Use **Specification pattern** or LINQ expressions



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

- Inject `DbContext` into repository if needed for advanced queries
- Optionally, break out complex queries into **Query objects** or **services**

```
public interface IProductRepository : IRepository<Product>
{
    Task<IEnumerable<Product>> GetProductsWithLowStockAsync(int
threshold);
}
```

20. What are potential drawbacks of Repository pattern?

Answer:

- **Over-abstraction:** Can add unnecessary complexity for simple apps.
- **Duplication:** May duplicate what EF Core already provides (since EF is already a repository/unit-of-work pattern).
- **Hides EF Core features:** May obscure advanced capabilities like eager loading or projections.
- **Extra boilerplate:** Especially with generic repositories, which may not add much value.

Unit of Work

21. What is the Unit of Work pattern?

Answer:

The **Unit of Work** pattern is a design pattern used to maintain a list of operations to be performed within a single transaction. It ensures that all operations either succeed or fail together, providing consistency and managing changes to multiple business objects during a transaction. It coordinates the writing out of changes and resolves potential concurrency issues.



22. How does Unit of Work complement the Repository pattern?

Answer:

The **Repository pattern** abstracts the data access layer, providing a simplified interface to data operations. The **Unit of Work** pattern complements it by managing multiple repositories and ensuring that all changes made through these repositories are committed in a single transaction. This combination separates concerns, promotes clean architecture, and maintains transactional integrity across multiple operations.

23. How do you implement Unit of Work in a .NET application?

Answer:

In a .NET application (especially using Entity Framework), the **Unit of Work** is typically implemented around the **DbContext**, as it already tracks changes and handles transactions. Here's a simplified example:

```
// IUnitOfWork.cs
public interface IUnitOfWork : IDisposable
{
    IProductRepository Products { get; }
    ICustomerRepository Customers { get; }
    int Complete();
}

// UnitOfWork.cs
public class UnitOfWork : IUnitOfWork
{
    private readonly AppDbContext _context;

    public IProductRepository Products { get; private set; }
    public ICustomerRepository Customers { get; private set; }

    public UnitOfWork(AppDbContext context)
    {
        _context = context;
        Products = new ProductRepository(_context);
    }
}
```



```
        Customers = new CustomerRepository(_context);
    }

    public int Complete()
    {
        return _context.SaveChanges(); // All changes in one
transaction
    }

    public void Dispose()
    {
        _context.Dispose();
    }
}
```

Then register it using Dependency Injection in `Startup.cs` or `Program.cs`:

```
services.AddScoped<IUnitOfWork, UnitOfWork>();
```

24. What are the benefits of Unit of Work in managing transactions?

Answer:

- **Transactional consistency:** All changes across repositories are saved in a single transaction.
- **Centralized commit:** All changes are committed through one method (`Complete()`), improving control and readability.
- **Reduced code duplication:** Prevents repeated save logic across repositories.
- **Improved testability:** Enables better unit testing by mocking a single Unit of Work instead of multiple repositories.
- **Change tracking:** Ensures that only modified entities are persisted, reducing unnecessary database operations.



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

25. Can you combine Unit of Work with Dependency Injection?

Answer:

Yes, combining Unit of Work with Dependency Injection (DI) is a best practice in modern .NET applications. DI allows the Unit of Work to be injected into services or controllers, managing its lifecycle effectively. This improves modularity, promotes loose coupling, and makes the application easier to maintain and test.

Example using constructor injection in a service:

```
public class OrderService
{
    private readonly IUnitOfWork _unitOfWork;

    public OrderService(IUnitOfWork unitOfWork)
    {
        _unitOfWork = unitOfWork;
    }

    public void PlaceOrder(Order order)
    {
        _unitOfWork.Orders.Add(order);
        _unitOfWork.Complete();
    }
}
```

Dependency Injection (DI)

26. What is Dependency Injection and why is it useful?

Answer:

Dependency Injection (DI) is a design pattern that allows an object to receive its dependencies from an external source rather than creating them itself. This promotes loose coupling, enhances testability, and simplifies code maintenance.



Benefits:

- Improves modularity
 - Enables easier unit testing (via mock dependencies)
 - Promotes adherence to SOLID principles (especially the Dependency Inversion Principle)
-

27. What are the different types of Dependency Injection?

Answer:

1. **Constructor Injection** – Dependencies are provided through the class constructor.
 2. **Property Injection** – Dependencies are set via public properties.
 3. **Method Injection** – Dependencies are passed as method parameters.
 4. **Service Locator (anti-pattern)** – Dependencies are resolved manually using a container, though generally discouraged.
-

28. How does .NET Core support Dependency Injection out of the box?

Answer:

.NET Core has a **built-in DI container** that's tightly integrated into the framework. When you create a new ASP.NET Core project, DI is configured automatically and used in controllers, middleware, services, etc. You register dependencies in the `Program.cs` or `Startup.cs` file using `IServiceCollection`.

29. How do you register services for DI in ASP.NET Core?

Answer:

Services are registered in the `Program.cs` or `Startup.cs` file using the `IServiceCollection` interface. Example:



```
builder.Services.AddTransient<IProductService, ProductService>();  
builder.Services.AddScoped<IOrderService, OrderService>();  
builder.Services.AddSingleton<ILoggingService, LoggingService>();
```

30. What is the difference between Scoped, Singleton, and Transient lifetimes?

Answer:

- **Transient:** A new instance is created **every time** it is requested.
- **Scoped:** A single instance is created **per request/HTTP request**.
- **Singleton:** A **single instance** is created and shared across the application **lifetime**.

Lifetime	Created Per	Use Case
Transient	Every injection	Lightweight, stateless services
Scoped	HTTP Request	Web services handling per-request data
Singleton	App lifetime	Logging, configuration, cache

31. How do you handle circular dependencies in DI?

Answer:

Circular dependencies occur when two or more services depend on each other, creating an infinite loop. To handle them:

- Refactor the code to remove tight coupling.
- Use interfaces or events to break the cycle.
- Consider using property injection carefully (not recommended as a first choice).
- Rethink your design – circular dependencies often indicate architectural issues.



32. What are the advantages of using DI in unit testing?

Answer:

- **Easier mocking** of dependencies using frameworks like Moq or NSubstitute.
- **No need to instantiate real implementations** (e.g., database access) for tests.
- **Improved isolation** of the unit under test.
- **Faster, more reliable tests** due to lack of external dependency reliance.

33. Can DI be used without a DI container? How?

Answer:

Yes, DI can be implemented **manually** without using a DI container. You simply pass dependencies through constructors or methods:

```
public class OrderService
{
    private readonly IOrderRepository _repo;

    public OrderService(IOrderRepository repo)
    {
        _repo = repo;
    }
}

// Manual injection
var repo = new OrderRepository();
var service = new OrderService(repo);
```

This is suitable for small or simple applications, though not scalable for large apps.



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

34. What are common pitfalls when implementing DI?

Answer:

- **Over-injection** (too many dependencies in one class – violates SRP)
- **Service locator anti-pattern**
- **Incorrect lifetimes**, leading to memory leaks or unintended reuse
- **Tight coupling to the DI container** (e.g., using container APIs in business logic)
- **Circular dependencies** due to poor design
- **Registering services in the wrong order or not at all**

35. How does DI relate to Inversion of Control (IoC)?

Answer:

Inversion of Control (IoC) is a broader principle where control over object creation and dependency resolution is transferred from the class itself to an external framework or container.

Dependency Injection is a specific implementation of IoC, where the dependencies are *injected* into a class rather than the class creating them internally. So, DI is one way to achieve IoC.

Single Responsibility Principle (SRP)

36. What is the Single Responsibility Principle?



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

Answer:

The **Single Responsibility Principle (SRP)** states that a class should have **only one reason to change**. In other words, it should have **only one responsibility or job**.

Each class should do one thing, and do it well.

For example, a class that handles user authentication should not also be responsible for logging or sending emails.

37. How can you identify violations of SRP in code?

Answer:

You can identify SRP violations by asking questions like:

- Does this class **perform more than one function** (e.g., data access and business logic)?
- Does it **change for different reasons** (e.g., changes in UI and database)?
- Does it **have too many dependencies or too much code**?
- Are there **“and”s in the class name or method descriptions**? E.g., `ReportGeneratorAndPrinter`.

Common signs:

- Long classes or large files
- Many unrelated methods
- Hard-to-test code

38. Can you give an example of refactoring code to follow SRP?

Answer:

Before (SRP Violation):

```
public class Invoice
```



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

```
{  
    public void GenerateInvoice() { /* logic */ }  
    public void SaveToDatabase() { /* logic */ }  
    public void SendEmail() { /* logic */ }  
}
```

This class has **3 responsibilities**: generating, saving, and emailing.

After (SRP-compliant):

```
public class InvoiceGenerator  
{  
    public void Generate() { /* logic */ }  
}  
  
public class InvoiceRepository  
{  
    public void Save(Invoice invoice) { /* logic */ }  
}  
  
public class EmailService  
{  
    public void Send(Invoice invoice) { /* logic */ }  
}
```

Now each class has **a single reason to change**.

39. How does SRP improve maintainability?

Answer:

SRP improves maintainability by:

- **Reducing complexity** – Classes are smaller and easier to understand.
- **Easier testing** – Each class can be tested in isolation.



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

- **Better separation of concerns** – Business logic, data access, and infrastructure code are kept apart.
- **Lower risk of bugs** – Changes in one responsibility don't affect others.

Ultimately, it leads to **more modular, flexible, and robust code**.

40. What happens if a class has multiple responsibilities?

Answer:

If a class has multiple responsibilities:

- **It becomes tightly coupled** and harder to change without affecting other parts.
- **Changes are more error-prone** and often introduce bugs.
- **Testing becomes harder** since the class relies on multiple behaviors.
- **Code reusability and readability suffer** due to mixed concerns.
- **You violate SRP**, which makes code harder to maintain in the long run.

Open/Closed Principle (OCP)

41. What does the Open/Closed Principle mean?

Answer:

The **Open/Closed Principle (OCP)** states that:

Software entities (classes, modules, functions) should be open for extension but closed for modification.

This means you should be able to **add new behavior** to a class **without changing its existing code**, which helps avoid breaking existing functionality.



42. How can you design classes that are open for extension but closed for modification?

Answer:

To design classes that follow OCP:

- **Use abstraction** (interfaces or abstract classes).
- Rely on **polymorphism** and **inheritance**.
- Apply **composition over inheritance** when suitable.
- Follow design principles like **Strategy**, **Decorator**, or **Template Method** patterns.

✓ Extend behavior through new classes rather than modifying existing code.

43. How do design patterns help in adhering to OCP?

Answer:

Several **design patterns** are built around the idea of making systems extensible without modifying core logic:

Pattern	How It Helps With OCP
Strategy	Allows changing behavior by swapping strategies.
Decorator	Adds new responsibilities dynamically without changing original code.
Template Method	Allows subclasses to override certain steps in an algorithm.
Factory Method	Makes it easy to introduce new types without altering existing logic.
Observer	Extends behavior in reaction to events without altering the source.

44. What is an example where OCP is violated?

Answer:

Before (OCP Violation):

```
public class DiscountCalculator
```



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

```
{  
    public decimal CalculateDiscount(string customerType)  
    {  
        if (customerType == "Regular") return 10;  
        if (customerType == "Premium") return 20;  
        if (customerType == "VIP") return 30;  
        return 0;  
    }  
}
```

If you need to support a new customer type, you must **modify** this method — violating OCP.

After (OCP Compliant):

```
public interface IDiscountStrategy  
{  
    decimal GetDiscount();  
}  
  
public class RegularCustomerDiscount : IDiscountStrategy  
{  
    public decimal GetDiscount() => 10;  
}  
  
public class PremiumCustomerDiscount : IDiscountStrategy  
{  
    public decimal GetDiscount() => 20;  
}  
  
public class VipCustomerDiscount : IDiscountStrategy  
{  
    public decimal GetDiscount() => 30;  
}
```

Now you can add new customer types **without modifying** existing code — just create a new strategy.



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

45. How does OCP relate to interfaces and abstract classes?

Answer:

- **Interfaces and abstract classes** provide a **contract** that other classes can implement or inherit.
- They **enable polymorphism**, which allows behavior to be extended without altering existing code.
- By **programming to abstractions** (not concrete implementations), you can easily introduce new behavior (via new implementations) while keeping the core logic unchanged.

✓ OCP encourages **extending via new subclasses or interface implementations**, not modifying existing ones.

Liskov Substitution Principle (LSP)

46. What is Liskov Substitution Principle?

Answer:

The **Liskov Substitution Principle (LSP)** states that:

Subtypes must be substitutable for their base types without altering the correctness of the program.

In other words, if class **S** is a subclass of class **T**, then objects of type **T** should be replaceable with objects of type **S** without breaking the application.

This ensures that **inheritance** models **is-a** relationships correctly.

47. How can violating LSP cause issues in software design?



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

Answer:

Violating LSP can lead to:

- **Unexpected behavior** when a subclass does not honor the contract of the base class.
- **Code that breaks at runtime** when substituting a derived class.
- **Tightly coupled code** that depends on specific implementations rather than abstractions.
- **Unit tests failing** when testing subclasses in place of base classes.

Essentially, it **defeats the purpose of polymorphism**.

48. What is an example of violating LSP in .NET code?

Answer:

Example (Violation of LSP):

```
public class Rectangle
{
    public virtual int Width { get; set; }
    public virtual int Height { get; set; }

    public int Area() => Width * Height;
}

public class Square : Rectangle
{
    public override int Width
    {
        set { base.Width = base.Height = value; }
    }

    public override int Height
    {
        set { base.Width = base.Height = value; }
    }
}
```



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

```
}  
}
```

Now if you substitute `Rectangle` with `Square`:

```
Rectangle rect = new Square();  
rect.Width = 5;  
rect.Height = 10;
```

```
Console.WriteLine(rect.Area()); // Outputs 100, but logically  
expected 50
```

✗ The behavior is **incorrect** — this is a **violation of LSP**.

49. How do you ensure subclasses follow LSP?

Answer:

To ensure subclasses follow LSP:

- Subclasses should **not override behavior** in a way that **breaks expected behavior**.
- Subclasses should **preserve the invariants** and preconditions/postconditions of the base class.
- Avoid **overriding methods** to throw exceptions for valid base class behavior.
- Use **composition over inheritance** if a subclass doesn't strictly conform to the base class behavior.
- Write **unit tests** to verify that the subclass behaves identically to the base class in all valid scenarios.

✓ Ask yourself: *Can this subclass be used anywhere the base class is used — without surprises?*



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

50. How does LSP relate to inheritance?

Answer:

LSP is fundamentally about **correct use of inheritance**. While inheritance allows code reuse, LSP ensures that the **behavior** of subclasses remains **consistent** with that of the base class.

If a subclass **changes the meaning** or **violates the expectations** of the base class's behavior, it's misusing inheritance.

Interface Segregation Principle (ISP)

51. What is Interface Segregation Principle?

Answer:

The **Interface Segregation Principle (ISP)** states that:

Clients should not be forced to depend on interfaces they do not use.

This means that interfaces should be **small and focused**, containing only the methods that are relevant to the implementing class. It prevents "fat" or "bloated" interfaces.

52. Why should interfaces be specific and small?

Answer:

Small, specific interfaces:

- Promote **separation of concerns**
- Make classes easier to **implement and test**
- Reduce the risk of **breaking changes**
- Avoid forcing classes to implement **irrelevant methods**
- Increase **reusability** and **readability**



In contrast, large interfaces force classes to implement methods they may not need — leading to fragile and cluttered code.

53. How does ISP improve code flexibility?

Answer:

ISP improves code flexibility by:

- Allowing classes to **only depend on what they actually use**
- Making it easier to **extend or replace functionality** without affecting unrelated parts
- Enabling **composition** over inheritance
- Making interfaces easier to **mock or stub in unit tests**
- Encouraging **clean, modular design**

Smaller interfaces result in **lower coupling** and **better maintainability**.

54. Can you provide an example of ISP violation?

Answer:

Violation Example:

```
public interface IWorker
{
    void Work();
    void Eat();
    void Sleep();
}

public class Robot : IWorker
{
    public void Work() { /* logic */ }
    public void Eat() { throw new NotImplementedException(); }
    public void Sleep() { throw new NotImplementedException(); }
```



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

```
}
```

✗ **Robot** is forced to implement **Eat()** and **Sleep()**, which don't make sense for it.

55. How do you refactor a fat interface to follow ISP?

Answer:

Refactored Using ISP:

```
public interface IWorkable
{
    void Work();
}

public interface IFeedable
{
    void Eat();
}

public interface ISleepable
{
    void Sleep();
}

public class Human : IWorkable, IFeedable, ISleepable
{
    public void Work() { }
    public void Eat() { }
    public void Sleep() { }
}

public class Robot : IWorkable
{
    public void Work() { }
}
```



✅ Now each class implements **only the interfaces it needs** — in line with ISP.

Inversion Principle (DIP)

56. What is Dependency Inversion Principle?

Answer:

The **Dependency Inversion Principle (DIP)** states that:

High-level modules should not depend on low-level modules. Both should depend on abstractions.

Abstractions should not depend on details. Details should depend on abstractions.

In other words:

- High-level business logic shouldn't depend on concrete implementations.
- Instead, both high- and low-level components should depend on **interfaces or abstract classes**.

57. How does DIP differ from Dependency Injection?

Answer:

Aspect	Dependency Inversion Principle (DIP)	Dependency Injection (DI)
Definition	A design principle about depending on abstractions	A technique for passing dependencies
Goal	Decouple high-level logic from low-level details	Provide dependencies to objects
Relation	DIP motivates the need for DI	DI is a way to implement DIP



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

Focus	What to depend on (abstractions)	How dependencies are supplied
-------	----------------------------------	-------------------------------

✓ DIP is a **design principle**, while DI is a **design pattern/technique** to implement that principle.

58. How do abstractions help in DIP?

Answer:

Abstractions (e.g., interfaces or abstract classes):

- **Decouple components** so changes in one don't ripple through others
 - Enable **substitution** of different implementations easily
 - Allow for **easier unit testing** with mocks/stubs
 - Promote **extensibility and maintainability**
 - Serve as **contracts** that both high- and low-level modules depend on
-

59. Can you explain DIP with an example in C#?

Answer:

✗ Without DIP (Tightly Coupled):

```
public class FileLogger
{
    public void Log(string message) => Console.WriteLine("File log: " + message);
}

public class OrderService
{
    private readonly FileLogger _logger = new FileLogger();
```



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

```
public void ProcessOrder()
{
    // Logic
    _logger.Log("Order processed.");
}
}
```

- `OrderService` is tightly coupled to `FileLogger`.

✓ With DIP (Loosely Coupled via Abstraction):

```
public interface ILogger
{
    void Log(string message);
}

public class FileLogger : ILogger
{
    public void Log(string message) => Console.WriteLine("File log: " + message);
}

public class OrderService
{
    private readonly ILogger _logger;

    public OrderService(ILogger logger)
    {
        _logger = logger;
    }

    public void ProcessOrder()
    {
        // Logic
        _logger.Log("Order processed.");
    }
}
```



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

```
}  
}
```

Now `OrderService` depends on the **abstraction** (`ILogger`), not the concrete `FileLogger`. You can easily substitute with `DatabaseLogger`, `ConsoleLogger`, or a mock in tests.

60. What are the benefits of following DIP?

Answer:

✓ Key Benefits:

- **Decouples components** — changes in low-level modules won't affect high-level ones
- **Improves testability** — you can easily inject mocks/stubs
- **Enhances flexibility** — swap implementations without touching core logic
- **Promotes reuse** — abstractions can be used across different modules
- **Supports SOLID architecture** — especially when combined with DI and IoC containers

Advanced & Scenario-Based Questions

61. How do design patterns help you follow SOLID principles?

Answer:

Design patterns provide structured, reusable solutions that embody SOLID principles. For example, the **Strategy pattern** supports OCP by allowing behavior extension without modifying existing code; **Repository** separates data access (SRP); **Dependency Injection**



supports DIP by decoupling high- and low-level modules. Using patterns helps keep code clean, modular, and maintainable.

62. Can you give an example where you combined Repository and Unit of Work patterns?

Answer:

The **Unit of Work** coordinates multiple **Repositories** and commits changes in a single transaction. For example:

```
public interface IUnitOfWork : IDisposable
{
    IProductRepository Products { get; }
    IOrderRepository Orders { get; }
    int Complete();
}

public class UnitOfWork : IUnitOfWork
{
    private readonly DbContext _context;
    public IProductRepository Products { get; }
    public IOrderRepository Orders { get; }

    public UnitOfWork(DbContext context)
    {
        _context = context;
        Products = new ProductRepository(_context);
        Orders = new OrderRepository(_context);
    }

    public int Complete() => _context.SaveChanges();

    public void Dispose() => _context.Dispose();
}
```

This ensures atomic commits across repositories.



63. How do you test classes that use Singleton or static instances?

Answer:

Testing singletons/statics is tricky due to global state. Best practices include:

- Refactor to use interfaces and DI instead of static/singletons.
- Wrap static calls behind interfaces so you can mock them.
- Use specialized mocking tools (e.g., Microsoft Fakes) if refactoring isn't possible.
- Avoid static state to improve testability.

64. What design pattern would you use to decouple a complex system and why?

Answer:

The **Mediator Pattern** centralizes communication between components, preventing direct dependencies and reducing complexity. It promotes loose coupling and simplifies interactions. Alternatively, the **Observer Pattern** enables event-driven decoupling, and **Facade Pattern** provides a simplified interface to complex subsystems.

65. How do you apply Strategy pattern to implement multiple payment methods?

Answer:

Define a common interface, e.g., `IPaymentStrategy` with a method `Pay()`. Implement concrete strategies for each payment method (CreditCard, PayPal, etc.). Use a context class to invoke the selected strategy at runtime, enabling easy extension without modifying existing code.

66. What is the difference between Repository and DAO patterns?



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

Answer:

DAO (Data Access Object) focuses on low-level database operations and CRUD, typically mapping tables to objects.

Repository abstracts data access at the domain level, working with aggregates/entities and encapsulating business logic. Repository often uses DAO internally but is more aligned with domain-driven design.

67. How do you refactor legacy code to follow SOLID principles?

Answer:

- Identify classes violating SRP and break them down.
 - Introduce abstractions and interfaces to decouple components (DIP).
 - Replace conditional logic with polymorphism to respect OCP.
 - Split large interfaces (ISP).
 - Check inheritance hierarchies to maintain LSP.
 - Inject dependencies instead of direct instantiation.
 - Incrementally refactor with unit tests to ensure behavior remains consistent.
-

68. How do you handle cross-cutting concerns (like logging) in a SOLID way?

Answer:

Use **AOP (Aspect-Oriented Programming)** techniques or design patterns like **Decorator** to separate cross-cutting concerns from business logic. In .NET, middleware, filters, or interceptors can manage concerns like logging or authorization, keeping SRP intact.

69. Can you explain how DI containers resolve dependencies at runtime?



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

Answer:

DI containers use **reflection** to inspect constructors of requested services, resolve dependencies recursively from their registrations, apply lifecycle scopes (Singleton, Scoped, Transient), and build the full object graph to return fully constructed instances.

70. How would you design a plugin architecture with SOLID principles?

Answer:

- Define plugin contracts with interfaces (DIP).
- Load plugins dynamically using reflection or MEF.
- Use DI to inject dependencies into plugins.
- Ensure plugins follow SRP with focused responsibilities.
- Use Factory or Strategy patterns to instantiate plugins.
- Keep core system closed for modification but open for extension (OCP).
- Separate cross-cutting concerns externally.

Practical .NET Questions

71. How do you implement DI in ASP.NET Core MVC?

Answer:

ASP.NET Core MVC has built-in support for Dependency Injection.

- Register services in `Startup.cs` within `ConfigureServices` method using `IServiceCollection`:

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddControllersWithViews();
}
```



```
services.AddScoped<IProductService, ProductService>(); //  
Example  
}
```

- Inject dependencies via constructor injection in controllers or services:

```
public class HomeController : Controller  
{  
    private readonly IProductService _productService;  
  
    public HomeController(IProductService productService)  
    {  
        _productService = productService;  
    }  
  
    public IActionResult Index()  
    {  
        var products = _productService.GetAll();  
        return View(products);  
    }  
}
```

The framework resolves and injects dependencies automatically.

72. What is the role of IServiceCollection and IServiceProvider in DI?

Answer:

- **IServiceCollection:** A container used during app startup to register services and their lifetimes (Scoped, Transient, Singleton). It acts as a **service registry**.
- **IServiceProvider:** The built container that **resolves** and **provides instances** of registered services at runtime. It uses the registrations to create and inject dependencies.



73. How do you configure EF Core Repository and Unit of Work in .NET Core?

Answer:

- Register `DbContext` with DI in `Startup.cs`:

```
services.AddDbContext<AppDbContext>(options =>
```

```
options.UseSqlServer(Configuration.GetConnectionString("DefaultConnection")));
```

- Implement Repositories for entities injecting `AppDbContext`.
- Implement Unit of Work which holds multiple repositories and calls `SaveChanges()` on the `DbContext`:

```
public interface IUnitOfWork : IDisposable
{
    IProductRepository Products { get; }
    int Complete();
}
```

```
public class UnitOfWork : IUnitOfWork
{
    private readonly AppDbContext _context;
    public IProductRepository Products { get; private set; }

    public UnitOfWork(AppDbContext context)
    {
        _context = context;
        Products = new ProductRepository(_context);
    }
}
```



```
public int Complete() => _context.SaveChanges();

public void Dispose() => _context.Dispose();
}
```

Register UnitOfWork in DI container as Scoped.

74. How do you mock dependencies in unit tests using DI?

Answer:

- Use mocking libraries like **Moq**, **NSubstitute**, or **FakeItEasy**.
- Create mocks of interfaces and inject them into the class under test:

```
var mockRepo = new Mock<IProductRepository>();
mockRepo.Setup(repo => repo.GetAll()).Returns(new List<Product> {
... });
```

```
var service = new ProductService(mockRepo.Object);
```

```
// Act & Assert
```

- This allows testing in isolation without hitting real databases or external services.
-

75. What design patterns are commonly used in ASP.NET Core middleware?

Answer:

- **Chain of Responsibility:** Middleware components form a pipeline where each decides to pass control or handle the request.



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

- **Decorator:** Middleware wraps around the next component, adding behavior before or after.
- **Factory:** Middleware components can be created via factories for configurable pipeline setup.

76. How do you apply the Strategy pattern to select different caching strategies in .NET?

Answer:

- Define a caching interface:

```
public interface ICacheStrategy
{
    void Cache(string key, object value);
    object Retrieve(string key);
}
```

- Implement strategies like `MemoryCacheStrategy`, `DistributedCacheStrategy`.
- Use DI or factory to inject the chosen strategy at runtime:

```
public class CacheContext
{
    private readonly ICacheStrategy _cacheStrategy;

    public CacheContext(ICacheStrategy cacheStrategy)
    {
        _cacheStrategy = cacheStrategy;
    }

    public void Cache(string key, object value) =>
        _cacheStrategy.Cache(key, value);
}
```



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

}

This enables switching caching mechanisms without code changes.

77. Can you explain the difference between Abstract Factory and Builder patterns with examples?

Answer:

- **Abstract Factory:** Provides an interface for creating families of related objects without specifying concrete classes.

Example: Creating UI components for different OS (Windows, Mac).

- **Builder:** Focuses on step-by-step construction of a complex object, allowing different representations.

Example: Building a complex **House** with various parts (walls, doors, roof).

Summary: Abstract Factory is about **families of products**, Builder is about **complex construction process**.

78. How do you avoid God classes in .NET applications?

Answer:

- Apply **Single Responsibility Principle (SRP)** by splitting responsibilities into smaller classes.
- Use **composition** instead of inheritance to delegate behavior.
- Extract business logic into services or helpers.
- Introduce abstractions to isolate concerns.
- Continuously refactor large classes and add unit tests.



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

79. What tools or libraries help you enforce SOLID principles in .NET code?

Answer:

- **Resharper**: Provides code analysis and refactoring hints.
- **SonarQube / SonarCloud**: Analyzes code quality and reports SOLID violations.
- **FxCop / Roslyn analyzers**: Provide static analysis with custom rules.
- **NDepend**: Deep architecture and dependency analysis tool.
- **StyleCop**: Enforces coding style which indirectly helps maintain SOLID code.

80. How does the Mediator pattern fit with SOLID and DI principles?

Answer:

- Mediator decouples components by centralizing communication, supporting **SRP** and **DIP** by reducing direct dependencies.
- It fits DI because the mediator itself can be injected where needed.
- It supports **OCP** by allowing new communication routes or handlers without modifying existing components.
- Helps avoid tight coupling in complex workflows or CQRS patterns.

Behavioral / Conceptual Questions



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

81. Describe a time when applying SOLID principles improved your project.

Answer:

In a recent project, we had a monolithic service class handling multiple responsibilities, making it hard to maintain and extend. By applying **Single Responsibility Principle (SRP)**, we split the class into focused services, each with a clear purpose. This drastically improved readability, reduced bugs, and made it easier to add new features without risking regressions. The project became more testable because each small class could be unit tested independently.

82. Have you ever faced issues with Singleton pattern in multi-threaded applications?

Answer:

Yes. In one project, a singleton was used without thread safety, causing race conditions when accessed concurrently. This led to inconsistent state and application crashes. We resolved it by implementing **thread-safe lazy initialization** using `Lazy<T>` in .NET, ensuring the singleton instance was created safely once, even under heavy parallel access.

83. How do you balance between over-engineering and following design principles?

Answer:

I prioritize **YAGNI (You Aren't Gonna Need It)** to avoid over-engineering. SOLID principles guide design for flexibility and maintainability, but I apply them pragmatically: start with simple solutions and refactor as requirements evolve. Writing tests early helps identify pain points justifying additional abstractions. Communication with the team ensures we don't add complexity prematurely but keep the codebase adaptable.

84. What challenges do you face when refactoring for SOLID compliance?

Answer:

Challenges include:



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

- **Legacy code with tight coupling**, making decomposition hard.
- Risk of **introducing bugs** while splitting responsibilities or introducing abstractions.
- Managing **dependencies and lifetimes** correctly when injecting dependencies.
- Convincing stakeholders that refactoring time is valuable.
- Balancing between adhering strictly to SOLID vs. keeping code understandable and performant.

85. How do you educate junior developers about design patterns and SOLID?

Answer:

I use a combination of:

- **Simple examples** showing before/after code refactoring.
- Pair programming sessions to explain thought processes.
- Encouraging reading and discussing classic books like *"Clean Code"* and *"Design Patterns"*.
- Practical coding exercises and code reviews focused on SOLID principles.
- Showing real project scenarios where principles improved code quality and maintainability.
- Promoting a culture of continuous learning and curiosity.

Bonus / Miscellaneous

86. What is the difference between Composition and Inheritance?



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

Answer:

- **Inheritance** creates an “is-a” relationship, where a subclass inherits behavior and properties from a parent class. It can lead to tight coupling and fragile hierarchies if overused.
 - **Composition** creates a “has-a” relationship, where a class contains instances of other classes and delegates behavior to them. It’s more flexible and promotes loose coupling.
-

87. How does the Decorator pattern differ from the Proxy pattern?

Answer:

- **Decorator** adds additional responsibilities to objects dynamically without altering their interface. It wraps the original object to extend behavior.
 - **Proxy** controls access to an object, possibly adding lazy initialization, access control, or logging, without changing its interface.
-

88. What is the Adapter pattern and how does it relate to SOLID?

Answer:

Adapter converts the interface of a class into another interface clients expect, allowing incompatible interfaces to work together. It promotes **Open/Closed Principle (OCP)** by enabling new integrations without modifying existing code.

89. How do you implement Lazy loading with design patterns?

Answer:

Use the **Proxy** or **Virtual Proxy** pattern where a placeholder object controls access to the real object and defers its creation until needed. In .NET, `Lazy<T>` provides built-in lazy loading.



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

90. What are anti-patterns related to DI and Singleton?

Answer:

- **Service Locator anti-pattern:** Hides dependencies instead of injecting them explicitly.
 - **Overusing Singleton:** Leads to hidden global state and testing difficulties.
 - **Improper Singleton thread safety:** Causes race conditions.
 - **Injecting concrete implementations:** Violates DIP.
-

91. Can you explain the Template Method pattern?

Answer:

Template Method defines the skeleton of an algorithm in a base class, deferring some steps to subclasses. It allows subclasses to redefine parts of the algorithm without changing its structure. It supports **OCP** by enabling extensions through inheritance.

92. How do SOLID principles help in microservices architecture?

Answer:

SOLID promotes small, single-responsibility services (SRP), clear interfaces (ISP), loose coupling (DIP), and extendable design (OCP). This aligns well with microservices by encouraging modular, maintainable, and testable service boundaries.

93. What design pattern helps in implementing Undo/Redo functionality?

Answer:

The **Command Pattern** encapsulates requests as objects, allowing operations to be stored, undone, or redone by maintaining command history.

94. How do you avoid tight coupling in large .NET projects?



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

Answer:

- Use **interfaces and abstractions** (DIP).
 - Apply **Dependency Injection**.
 - Modularize code into bounded contexts or separate projects.
 - Use **events or messaging** for decoupled communication.
 - Avoid static state and global variables.
-

95. How does the Command pattern work?

Answer:

Encapsulates a request as an object with methods to execute and possibly undo the operation. The invoker calls commands without knowing the action details, supporting decoupling and flexible request handling.

96. What is the difference between Cohesion and Coupling?

Answer:

- **Cohesion:** Degree to which elements of a module belong together. High cohesion means focused, well-defined responsibilities.
 - **Coupling:** Degree of interdependence between modules. Low coupling means modules are independent and changes in one don't affect others.
-

97. How do you manage dependencies between microservices?

Answer:

- Use **API contracts** and versioning.



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

- Employ **service discovery** and **load balancing**.
- Prefer **asynchronous messaging/event-driven architecture** to reduce tight coupling.
- Apply **circuit breakers** and retries for fault tolerance.

98. What is the role of Abstract classes vs Interfaces in SOLID design?

Answer:

- **Interfaces** define contracts with no implementation, supporting **ISP** and **DIP** by allowing flexible implementations.
- **Abstract classes** provide a base implementation and shared code, useful for **Template Method** pattern and partial abstraction.

99. Can you explain the Builder pattern with a real-world .NET example?

Answer:

Builder separates complex object construction from its representation. For example, building an `HttpRequest` with optional headers, query params, and body:

```
public class HttpRequestBuilder
{
    private HttpRequestMessage _request = new HttpRequestMessage();

    public HttpRequestBuilder SetMethod(HttpMethod method)
    {
        _request.Method = method;
        return this;
    }

    public HttpRequestBuilder AddHeader(string key, string value)
    {

```



SANDEEP PAL

FULL STACK DEVELOPER (DOTNET, CLOUD, API & REACT)

@Sandeep_pall

```
        _request.Headers.Add(key, value);  
        return this;  
    }  
  
    public HttpRequestMessage Build() => _request;  
}
```

Allows building requests step-by-step fluently.

100. How does the Observer pattern fit into SOLID and DI?

Answer:

Observer supports **SRP** by separating notification logic from core business logic and **DIP** by depending on abstractions (observers). It fits DI because observers can be injected, allowing flexible runtime subscriptions and loose coupling.